

- RAG Learning

4.5 Fundamentals of Data Representation

Computer Science — AQA A-Level (7517) — Paper 2: Written Theory

Topic 4.5 is the mechanical heart of the specification. Everything a computer stores — integers, fractions, characters, pixels, samples of sound, compressed streams, ciphertext — is ultimately a finite string of bits. This note walks sections 4.5.1–4.5.13 in spec order, with worked bit-by-bit conversions, IEEE-754 layouts, Huffman constructions and Vernam cipher examples that mirror the style of AQA mark schemes. Where the examiners award marks for *showing working*, the working is shown in full.

4.5.1 Number Systems

Before choosing a representation, one must decide *what kind of number* is being represented. The spec enumerates five overlapping sets of numbers plus the notion of ordinal position.

Set	Symbol	Contents	Example
Natural	$\mathbb{N}$	Non-negative counting numbers.	0, 1, 2, 3, 17, 42
Integer	$\mathbb{Z}$	Naturals plus their negatives.	−7, −1, 0, 4, 99
Rational	$\mathbb{Q}$	Ratios p/q of integers ($q \neq 0$).	$3/4$, −5, 0.125, $2/7$
Irrational	—	Real numbers that cannot be written as p/q .	$\sqrt{2}$, π , e
Real	$\mathbb{R}$	Rationals $\cup$ irrationals — the continuum.	Any point on the number line

Two conceptual distinctions follow.

- **Counting vs measurement.** Counting yields naturals (how many apples?). Measurement yields reals (how tall is the tree?). Computers must round or truncate every measurement because they have only finite bits.
- **Cardinal vs ordinal.** A *cardinal* number answers "how many" (there are 7 pupils). An *ordinal* number answers "in what position" (she came 3rd). Array indexing is ordinal — `a[0]` means "the first element", not "zero elements".

Ordinal number — a number used to indicate position in an ordered sequence (1st, 2nd, 3rd...). In computing, indices into arrays, ranks in a sorted list, and line numbers are ordinal.

4.5.2 Number Bases

A **number base** (or *radix*) is the number of distinct digits used to write values. Each column of a positional numeral is a successive power of the base.

- **Decimal (base 10)** — digits 0–9. Column weights ... 10^3 , 10^2 , 10^1 , 10^0 .
- **Binary (base 2)** — digits 0 and 1. Column weights ...8, 4, 2, 1. The native language of hardware.
- **Hexadecimal (base 16)** — digits 0–9 then A(10), B(11), C(12), D(13), E(14), F(15). One hex digit encodes exactly *four* binary digits, making hex a compact shorthand for binary.

Decimal → Binary (repeated division by 2)

Convert 173_{10} to binary. Divide by 2 repeatedly, reading remainders from bottom to top.

```

173 ÷ 2 = 86 remainder 1    (LSB)
 86 ÷ 2 = 43 remainder 0
 43 ÷ 2 = 21 remainder 1
 21 ÷ 2 = 10 remainder 1
 10 ÷ 2 =  5 remainder 0
  5 ÷ 2 =  2 remainder 1
  2 ÷ 2 =  1 remainder 0
  1 ÷ 2 =  0 remainder 1    (MSB)

```

Reading upwards: 1 0 1 0 1 1 0 1
 Answer: $173_{10} = 10101101_2$

Check by summing column weights: $128 + 32 + 8 + 4 + 1 = 173$. Correct.

Binary → Hex (group into nibbles)

Group bits in fours from the right, then look up each nibble.

10101101 → 1010 1101 → A D → 0xAD

Check: $0xAD = 10 \times 16 + 13 = 160 + 13 = 173$. Correct.

Fractional binary

Columns *after* the binary point have weights $\frac{1}{2}$, $\frac{1}{4}$, $\frac{1}{8}$, $\frac{1}{16}$... To convert a decimal fraction to binary, multiply by 2 repeatedly and read the integer parts top-to-bottom.

Convert 0.6875 to binary:

$$0.6875 \times 2 = 1.375 \rightarrow 1$$

$$0.375 \times 2 = 0.75 \rightarrow 0$$

$$0.75 \times 2 = 1.5 \rightarrow 1$$

$$0.5 \times 2 = 1.0 \rightarrow 1 \quad (\text{fraction now zero – stop})$$

Answer: $0.6875_{10} = 0.1011_2$

Check: $\frac{1}{2} + \frac{1}{8} + \frac{1}{16} = 0.5 + 0.125 + 0.0625 = 0.6875$. Correct.

Exam Tip: When asked "convert this decimal to binary", always *show every division* and circle the remainder column. Mark schemes usually award 1 mark for method and 1 mark for the final answer — a bare answer scores only 1/2.

4.5.3 Units of Information

Bit — a *binary digit*, 0 or 1. The smallest unit of information a computer can store.

- **Nibble** — 4 bits. Encodes exactly one hex digit.

- **Byte** — 8 bits. The standard addressable unit of memory. Can hold one ASCII character, one unsigned integer 0–255, or two hex digits.

Binary (IEC) vs decimal (SI) prefixes

Historically "kilobyte" was used for both 1000 and 1024 bytes. The 1998 IEC standard separated the two. **Decimal (SI) prefixes** use powers of 10; **binary (IEC) prefixes** use powers of 2.

Decimal name	Symbol	Value	Binary name	Symbol	Value
kilobyte	kB	$10^3 = 1000 \text{ B}$	kibibyte	KiB	$2^{10} = 1024 \text{ B}$
megabyte	MB	10^6 B	mebibyte	MiB	$2^{20} = 1048576 \text{ B}$
gigabyte	GB	10^9 B	gibibyte	GiB	2^{30} B
terabyte	TB	10^{12} B	tebibyte	TiB	2^{40} B

The confusion is practical: disk manufacturers quote capacity in *decimal* GB, while operating systems often report storage in *binary* GiB, so a "1 TB" disk shows as about 931 GiB in Windows. AQA expects candidates to know both families and to use IEC symbols when powers of 2 are intended.

4.5.4 The Binary Number System

Unsigned binary treats all bits as positive column values. An n -bit unsigned integer ranges from 0 to $2^n - 1$.

Binary addition

A	B	Sum	Carry out
0	0	0	0
0	1	1	0

1	0	1	0
1	1	0	1

Example: 01011101 + 00110110.

```

carries  1 1 1 1 1 1
          0 1 0 1 1 1 0 1    (93)
+         0 0 1 1 0 1 1 0    (54)
-----
          1 0 0 1 0 0 1 1    (147)

```

Subtraction by two's complement

To compute $A - B$ in binary, add A to the two's complement of B (see 4.5.6). Any carry out of the most-significant bit is discarded.

Shifts

- **Logical shift left by n** — bits move left, zeros fill the right. Equivalent to multiplying by 2^n (if no overflow).
- **Logical shift right by n** — bits move right, zeros fill the left. Equivalent to unsigned divide by 2^n (integer division).
- **Arithmetic shift right** — copies the sign bit into vacated left positions, preserving signed value. A signed divide by 2^n (rounding towards $-\infty$).

```

00010110  (22)
LSL 2 → 01011000  (88)    = 22 × 4

11001000  (-56 as 8-bit two's complement)
ASR 1 → 11100100  (-28)    = -56 ÷ 2
LSR 1 → 01100100  ( 100)    - sign lost, wrong for signed!

```

4.5.5 Information Coding Systems

ASCII

ASCII (American Standard Code for Information Interchange) — a 7-bit character set defining 128 code points (0–127) covering control codes, digits, upper and lower case English letters and common punctuation. Each character fits in one byte with a leading 0.

Character	Decimal	Hex	Binary
NUL	0	0x00	0000000
LF (newline)	10	0x0A	0001010
space	32	0x20	0100000
'0'	48	0x30	0110000
'9'	57	0x39	0111001
'A'	65	0x41	1000001
'a'	97	0x61	1100001
DEL	127	0x7F	1111111

Note that **'A' and 'a' differ by exactly 32** (bit 5) — flipping that bit converts case. Digits '0'–'9' are 48–57, so $'7' - '0' = 7$ converts a character digit to its numeric value (see 4.5.10).

Unicode and UTF-8

Unicode — a universal character set assigning a unique *code point* (U+0000 to U+10FFFF) to every character in every written language, plus mathematical symbols and emoji.

UTF-8 is the dominant *encoding* of Unicode. It uses 1 to 4 bytes per code point:

- U+0000–U+007F: 1 byte — byte-compatible with ASCII.
- U+0080–U+07FF: 2 bytes (Latin-supplement, Greek, Cyrillic, Arabic...).

- U+0800–U+FFFF: 3 bytes (CJK, most other scripts).
- U+10000–U+10FFFF: 4 bytes (rare historical scripts, emoji).

Advantages of Unicode over ASCII: global text support, unambiguous exchange of documents between languages, one encoding serves both English and e.g. Japanese in the same file, backward-compatible with ASCII (every ASCII file is valid UTF-8).

4.5.6 Representing Signed Integers

Two schemes are on the spec: sign-and-magnitude and two's complement. Two's complement is vastly more important because hardware addition "just works" across positive and negative numbers.

Sign and magnitude

The most-significant bit is a sign flag (0 = positive, 1 = negative); the remaining bits are the magnitude. Range of n -bit sign-magnitude is $-(2^{n-1}-1)$ to $+(2^{n-1}-1)$. It has two zeros (+0 and -0) and requires separate addition logic for the sign, so it is rarely used.

Two's complement

Two's complement — a signed binary representation in which the most-significant bit carries a *negative* weight. For 8-bit two's complement, the weights are -128, 64, 32, 16, 8, 4, 2, 1. Range: -2^{n-1} to $2^{n-1}-1$. There is exactly one zero, and ordinary binary addition produces the correct signed result.

To negate an n -bit two's-complement value: invert every bit, then add 1.

Example: write -42 in 8-bit two's complement.

Step 1	+42 in binary	=	0 0 1 0 1 0 1 0
Step 2	Invert all bits	=	1 1 0 1 0 1 0 1
Step 3	Add 1	=	1 1 0 1 0 1 1 0

Check using negative MSB weights:

$$-128 + 64 + 16 + 4 + 2 = -128 + 86 = -42. \text{ Correct.}$$

Example: write -5 in 8-bit two's complement (for reference).

$$\begin{array}{rcl} +5 & = & 0\ 0\ 0\ 0\ 0\ 1\ 0\ 1 \\ \text{Invert} & = & 1\ 1\ 1\ 1\ 1\ 0\ 1\ 0 \\ \text{Add 1} & = & 1\ 1\ 1\ 1\ 1\ 0\ 1\ 1 \rightarrow -5 \end{array}$$

Four-bit two's-complement reference table

Bits	Unsigned	Two's complement
0000	0	0
0001	1	+1
0111	7	+7
1000	8	−8
1001	9	−7
1100	12	−4
1111	15	−1

Exam Tip: If asked to "convert $-X$ to n -bit two's complement", write the *positive* X in n bits first, then flip-and-add-one. Always show each of the three lines (positive, inverted, +1). Leaving off the "+1" is the single most common student error in 4.5.6.

4.5.7 Representing Fractions in Binary

Fixed-point

The binary point is at a pre-agreed position. For example 8-bit fixed-point with 4 fractional bits interprets `01011010` as $0101.1010_2 = 5.625_{10}$. Simple, but range and precision are inflexible: large numbers sacrifice precision and vice versa.

Floating-point

Floating-point — a representation of real numbers as $\pm m \times 2^e$, storing a signed **mantissa** m and a signed **exponent** e . The point "floats" — large values use a big exponent, small values a small (possibly negative) exponent — giving a much wider dynamic range than fixed-point for the same number of bits.

Mantissa (significand) — the signed fractional component that carries the precision of a floating-point number.

Exponent — the signed power of 2 that scales the mantissa in a floating-point number.

IEEE 754 single precision (32-bit)

S	Exponent (8 bits)		Mantissa (23 bits)	
1	b7 b6 b5 b4 b3 b2 b1 b0		m22 m21 m20 ... m2 m1 m0	

Value = $(-1)^S \times 1.\text{mantissa} \times 2^{(\text{exponent} - 127)}$

S = sign bit (0 positive, 1 negative)

Exponent = 8-bit biased integer (bias = 127)

Mantissa = 23-bit fractional part of a normalised value 1.xxxx
(the leading 1 is *implicit* — not stored)

AQA-style variant

AQA exam questions typically use a simpler format: a two's-complement mantissa with an assumed binary point after bit 1 (e.g. `0.1xxxxxx`), and a two's-complement exponent — no bias, no hidden bit. The conversion logic is identical: normalise, then split into mantissa and exponent.

Normalisation — adjusting a floating-point number so its mantissa has the maximum precision. In two's complement the rule is: the first two bits must differ ($0.1...$ for positives, $1.0...$ for negatives). This guarantees no leading redundant sign bits — every bit contributes useful precision.

Worked example: IEEE 754 representation of -6.25

```

Step 1   $|-6.25| = 6.25 = 110.01_2$ 
Step 2  Normalise:       $110.01 = 1.1001 \times 2^2$ 
Step 3  Sign:            $S = 1$  (negative)
Step 4  Exponent:        $2 + 127 = 129 = 10000001_2$ 
Step 5  Mantissa:       drop the leading 1, pad to 23 bits
                        1001 000 0000 0000 0000 0000

```

```

Result (32 bits):
  1 10000001 100100000000000000000000
  = 0xC0C80000

```

Converting back

Given $0\ 10000010\ 010000000000000000000000$: $S = 0$ (positive); exponent $10000010 = 130$, minus bias $127 = 3$; mantissa (with hidden 1) $= 1.01_2 = 1.25$. Value $= +1.25 \times 2^3 = +10.0_{10}$.

Precision, accuracy and error

- **Precision** — the number of significant bits in the mantissa. More mantissa bits $\Rightarrow$ values can be expressed more finely.
- **Accuracy** — how close the stored value is to the true value. Limited precision means most real numbers are only *approximated*.
- **Absolute error** = $|\text{true value} - \text{stored value}|$.
- **Relative error** = $\text{absolute error} / |\text{true value}|$. Usually the more informative measure — an error of 0.01 matters on a value near 1 but not on a value near 10^6 .
- Rounding modes: *round to nearest* (IEEE default, ties to even), *truncation* (toward zero), *round up*, *round down*. Repeated rounding in long calculations causes *accumulated error*.

Exam Tip: To "show a number is normalised", point to the first two bits and state that they differ (0.1 for positive, 1.0 for negative). Examiners specifically look for a justification — not just the phrase "it is normalised".

4.5.8 Floating-Point Arithmetic

Addition of two floats proceeds in four steps: **align exponents, add mantissas, renormalise, round if necessary.**

Worked example

Add the AQA-style numbers $X = 0.1010 \times 2^{011}$ and $Y = 0.1100 \times 2^{001}$ (mantissas in two's complement, exponents in two's complement, 4 mantissa bits, 3 exponent bits).

Step 1 Decode

$$\begin{aligned} X &= 0.1010_2 \times 2^3 = 0.625 \times 8 = 5 \\ Y &= 0.1100_2 \times 2^1 = 0.75 \times 2 = 1.5 \end{aligned}$$

Step 2 Align exponents (shift Y's mantissa right until its exponent = 3)
Y: shift right by 2 $\Rightarrow$ mantissa 0.0011, exponent 011
(bits falling off the right are lost – precision loss)

Step 3 Add mantissas

$$\begin{array}{r} 0.1010 \\ + 0.0011 \\ \hline 0.1101 \end{array} \quad \text{exponent remains } 011$$

Step 4 Result before renormalisation: 0.1101×2^3
First two bits already differ $\Rightarrow$ already normalised.

Step 5 Decode to check

$$0.1101_2 \times 2^3 = 0.8125 \times 8 = 6.5.$$

Expected $5 + 1.5 = 6.5$. Correct.

If the addition produced a carry (e.g. 1.0101), we would shift the mantissa *right* by 1 and increment the exponent by 1 to renormalise. If leading bits matched (e.g. 0.0110), we would shift *left* and decrement the exponent.

Precision loss

Aligning exponents discards low-order mantissa bits of the smaller operand; adding two nearly-equal numbers of opposite sign (catastrophic cancellation) loses leading bits; and repeated rounding accumulates error across long sums. These are intrinsic to finite-precision floating-point, not bugs.

4.5.9 Bitwise Manipulation and Masks

A **mask** is a bit-pattern combined with data via a bitwise operator to isolate, set, clear or flip selected bits.

Operation	Symbol	Effect	Typical use
AND	&	$1 \& x = x$; $0 \& x = 0$	<i>Clear</i> bits (mask with 0) or <i>test</i> bits (read only chosen ones).
OR		$1 x = 1$; $0 x = x$	<i>Set</i> bits (mask with 1 in chosen positions).
XOR	^	$x \wedge 0 = x$; $x \wedge 1 = \neg x$	<i>Toggle</i> bits; used in Vernam/one-time-pad encryption.
NOT	~	Inverts every bit.	Build an inverse mask.

Worked examples

Data	1 0 1 1 0 1 0 1
AND mask (keep low nibble)	0 0 0 0 1 1 1 1

Result	0 0 0 0 0 1 0 1
Data	1 0 1 1 0 1 0 1

OR mask (force top two bits)	1 1 0 0 0 0 0 0

Result	1 1 1 1 0 1 0 1
Data	1 0 1 1 0 1 0 1
XOR mask (toggle bits 0,2,4)	0 0 0 1 0 1 0 1

Result	1 0 1 0 0 0 0 0

Shifts reappear here: `x << n` multiplies an unsigned `x` by 2^n ; `x >> n` divides (integer). Arithmetic right-shift preserves sign on two's-complement values; logical right-shift does not.

4.5.10 Character Form of a Decimal Digit

The character `'7'` is *not* the integer `7`. `'7'` is stored as ASCII 55 (0x37), not 7. The character `'0'` is 48, so:

<code>digit_value</code>	<code>= ord(ch) - ord('0')</code>	# e.g. <code>ord('7') - 48 = 7</code>
<code>character</code>	<code>= chr(n + ord('0'))</code>	# e.g. <code>chr(3 + 48) = '3'</code>

This distinction matters whenever input/output is textual. Reading the string "173" from a keyboard gives three bytes 49, 55, 51 — a program that needs the integer 173 must iterate, subtract 48, and accumulate `total = total×10 + digit`.

4.5.11 Representing Images, Sound and Other Data

Bitmap (raster) images

A bitmap stores a rectangular grid of *pixels*. Three parameters dominate:

- **Resolution** — width × height in pixels (e.g. 800 × 600).
- **Colour depth** (bits per pixel) — 1 (mono), 8 (256 colours/greyscale), 24 (true colour = 8 bits each R,G,B), 32 (true colour + alpha).

- **Metadata** — header fields recording width, height, colour depth, compression used, colour palette etc.

Uncompressed file size in bits = $width \times height \times bit\ depth$.

Example: 800×600 , 24-bit true colour

Pixels	= 800×600	= 480 000
Bits	= $480\ 000 \times 24$	= 11 520 000 bits
Bytes	= $11\ 520\ 000 / 8$	= 1 440 000 B
KiB	= $1\ 440\ 000 / 1024$	$\approx 1406.25\ KiB \approx 1.37\ MiB$

Doubling the resolution in both dimensions quadruples the file size.

Vector graphics

Vector images store *drawing instructions* — shape type, coordinates, stroke, fill — rather than pixels. Advantages: resolution-independent scaling without loss; tiny files for simple illustrations; easy editing of individual objects. Disadvantages: unsuitable for photos; rendering time grows with shape count.

Sound

Sound is an analogue waveform — a continuously varying pressure. To store it digitally we *sample* the amplitude at regular intervals and quantise each sample to an integer.

- **Sample rate (sampling frequency)** — samples taken per second, in Hz. CD audio uses 44.1 kHz.
- **Sample resolution (bit depth)** — bits used per sample. CD audio uses 16 bits.
- **Nyquist–Shannon theorem** — to reproduce a signal whose highest frequency is f , the sampling rate must be at least $2f$. Human hearing tops out around 20 kHz, so 44.1 kHz sampling captures the full audible spectrum with a small safety margin.

Uncompressed PCM mono file size (bits) = $duration_s \times sample_rate \times bit_depth$

Example: 30 s of CD-quality mono

= $30 \times 44100 \times 16$ = 21 168 000 bits = 2 646 000 B $\approx 2.52\ MiB$

Stereo doubles it; 5.1 surround multiplies by six.

Analogue-to-digital conversion (ADC)

Real-world sensors (microphone, light sensor, thermometer) produce analogue signals. An ADC samples the voltage at the chosen rate and outputs the nearest quantisation level as an integer. Higher sample rate captures higher frequencies (Nyquist); higher bit depth reduces quantisation error.

Other data

Video is a sequence of bitmap frames plus audio; documents are structured byte streams (PDF, DOCX); sensor streams are time-series of ADC samples. All are reducible to bytes — the representation chosen determines file size, fidelity and ease of processing.

4.5.12 Data Compression

Compression reduces the number of bits needed to store or transmit data. It exploits redundancy — repeated symbols, patterns or imperceptible detail.

- **Lossless** — decompression reconstructs the original bit-for-bit. Suitable for text, source code, executables, archival master files.
- **Lossy** — discards information judged unimportant to human perception. Far higher compression ratios, but the original cannot be recovered. Suitable for photographs, music, video.

Lossless compression — any encoding scheme for which decompressing the output yields the exact original data. Examples include Run-Length Encoding, Huffman coding, and Lempel–Ziv dictionary methods.

Format	Media	Lossy or lossless
PNG	Image	Lossless
GIF	Image	Lossless (palette-limited)

JPEG	Image	Lossy
FLAC	Audio	Lossless
MP3 / AAC	Audio	Lossy
ZIP / GZIP	General	Lossless
H.264 / HEVC	Video	Lossy

Run-length encoding (RLE)

Replace each run of identical symbols with the pair (symbol, run length). Works well on data with long uniform stretches (e.g. simple graphics, fax images); worthless on random or high-entropy data where it can even *expand* the file.

```
Original:   AAAAABBBCCDAAA
RLE output: (A,5)(B,3)(C,2)(D,1)(A,3)
            ⇒ 5 pairs × 2 bytes = 10 bytes
            – versus 14 bytes uncompressed.
```

Huffman coding

Huffman assigns *variable-length* binary codes to symbols so frequent symbols receive short codes and rare symbols receive long codes. The resulting code is a *prefix code* — no code is a prefix of another — so decoding is unambiguous without delimiters.

Worked example. Frequency table {A:5, B:2, R:2, C:1, D:1}. Total = 11 symbols.

Algorithm: repeatedly take the two lowest-frequency nodes, merge them into a parent whose frequency is their sum, put the parent back. Continue until one root remains.

```
Start list of (freq, symbol) nodes:
  [ (1,C) (1,D) (2,B) (2,R) (5,A) ]

Step 1 Merge C(1) + D(1) → node CD(2)
  [ (2,B) (2,R) (2,CD) (5,A) ]
```

Step 2 Merge B(2) + R(2) → node BR(4)

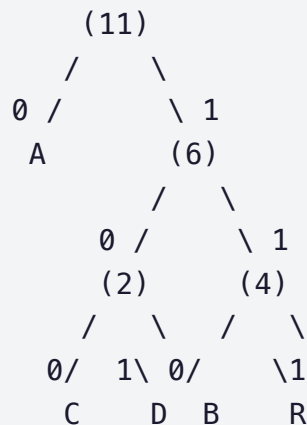
[(2,CD) (4,BR) (5,A)]

Step 3 Merge CD(2) + BR(4) → node CDBR(6)

[(5,A) (6,CDBR)]

Step 4 Merge A(5) + CDBR(6) → root(11)

Tree (left edge = 0, right edge = 1):



Codes:

A = 0

C = 100

D = 101

B = 110

R = 111

Total bits to encode "ABRACADABRA" (A×5, B×2, R×2, C×1, D×1):

$5 \times 1 + 2 \times 3 + 2 \times 3 + 1 \times 3 + 1 \times 3 = 23$ bits

(versus $11 \times 8 = 88$ bits in ASCII ⇒ ~74% smaller)

The exact codes depend on which child is labelled 0 vs 1 and which of two equal-frequency nodes is picked first, so different valid trees can exist. Examiners accept any tree built by the correct algorithm whose total bit-count matches.

Dictionary / Lempel–Ziv methods

Dictionary compressors (LZ77, LZ78, LZW, DEFLATE used in ZIP/gzip/PNG) build a table of repeated substrings seen earlier in the stream and replace later occurrences with short references

to the table. They adapt to the data and need no pre-agreed frequency table, making them the universal choice for general-purpose archive formats.

4.5.13 Encryption

Encryption transforms plaintext into ciphertext under the control of a *key*. Only holders of the correct key can decrypt.

Caesar cipher

A *substitution cipher* shifting every letter by a fixed key k through the alphabet (modulo 26).

```

FUNCTION caesarEncrypt(plaintext, k)
    ciphertext ← ""
    FOR EACH ch IN plaintext
        IF ch is a letter THEN
            base ← ord('A') (if upper) or ord('a') (if lower)
            ciphertext ← ciphertext + chr( (ord(ch) - base + k) MOD 26 + base )
        ELSE
            ciphertext ← ciphertext + ch
        ENDIF
    ENDFOR
    RETURN ciphertext

```

Example plaintext = "HELLO", k = 3
 H→K E→H L→O L→O O→R ⇒ "KH00R"

The Caesar cipher has only 25 useful keys and is trivially broken by frequency analysis or exhaustive search. It survives only as a teaching example.

Vernam cipher (one-time pad)

Vernam cipher — an encryption scheme in which each bit (or character) of plaintext is combined by XOR with the corresponding bit of a random key of equal length. If the key is *truly random, at least as long as the plaintext, used only once, and kept completely secret*, the Vernam cipher is **information-theoretically secure** (Shannon, 1949) — no amount of

computing power can break it, because every possible plaintext of the right length is equally consistent with the ciphertext.

Worked 8-bit example. Plaintext byte 0x48 ('H'); key byte 0xA5 (random).

```

Plaintext  P = 0 1 0 0 1 0 0 0      (0x48 = 'H')
Key        K = 1 0 1 0 0 1 0 1      (0xA5)
-----
Ciphertext C = 1 1 1 0 1 1 0 1      (0xED)    <-- P XOR K

Decryption: C XOR K = P
            C = 1 1 1 0 1 1 0 1
            K = 1 0 1 0 0 1 0 1
            -----
            P = 0 1 0 0 1 0 0 0      (0x48 = 'H')  ✓

```

The four conditions really matter: if the key is pseudo-random the scheme inherits the PRNG's weaknesses; if reused across two messages, XORing the two ciphertexts cancels the key and leaks $P1 \oplus P2$; if the key is shorter than the plaintext (a *stream cipher* such as RC4) the scheme stops being information-theoretically secure and relies on computational assumptions.

Symmetric vs asymmetric encryption

Feature	Symmetric	Asymmetric
Keys	One shared secret key	Key pair: public + private
Encrypt with	Shared key	Recipient's <i>public</i> key
Decrypt with	Shared key	Recipient's <i>private</i> key
Speed	Fast (bulk data)	Slow (100–1000× slower)
Key distribution	Hard — must share secretly	Easy — public keys can be posted openly
Examples	AES, Vernam, DES	RSA, ECC, Diffie-Hellman

In practice systems combine the two (*hybrid encryption*): use asymmetric RSA/ECDH to exchange a fresh random AES key, then encrypt bulk traffic symmetrically with AES. This is how TLS, SSH and HTTPS operate.

Hash functions

Hash function — a deterministic function h : arbitrary-length bytes $\rightarrow$ fixed-length digest (e.g. 256 bits). For a **cryptographic** hash it must be: *one-way* (given $h(x)$, infeasible to find x); *collision-resistant* (infeasible to find $x \neq y$ with $h(x) = h(y)$); and *avalanche* (a 1-bit change to x scrambles the whole digest).

Uses:

- **Integrity checking** — publish $h(\text{file})$; downloaders re-hash their copy and compare.
- **Digital signatures** — sign the hash, not the whole document; verifier re-hashes and checks the signature against the public key.
- **Password storage** — store $h(\text{password} + \text{salt})$, never the password itself; on login hash the input and compare. If the database leaks, the passwords remain hidden.
- **Blockchains and Merkle trees** — every block contains the hash of its parent, so any tampering propagates forward and is detectable.

Widely used cryptographic hashes include SHA-256 and SHA-3. MD5 and SHA-1 are now considered broken because collisions have been demonstrated in practice.

Exam Tip: If asked "why is the Vernam cipher perfectly secure?" you must state all four conditions — key is *truly random*, *at least as long* as the plaintext, used *only once*, kept *completely secret* — and explain that under these conditions every possible plaintext is equally likely given the ciphertext, so no information is leaked. Missing even one condition costs a mark.

Summary of spec coverage

- 4.5.1 Number systems — naturals, integers, rationals, irrationals, reals, ordinals.

- 4.5.2 Number bases — binary, decimal, hex, conversions including fractions.
- 4.5.3 Units — bit, nibble, byte; SI vs IEC prefixes.
- 4.5.4 Unsigned binary — addition, subtraction, logical and arithmetic shifts.
- 4.5.5 Coding systems — ASCII, Unicode, UTF-8.
- 4.5.6 Signed integers — sign-magnitude, two's complement, range, negation.
- 4.5.7 Fractions — fixed-point, IEEE 754, mantissa, exponent, normalisation, error.
- 4.5.8 Floating-point arithmetic — align, add, renormalise, precision loss.
- 4.5.9 Bitwise masks — AND clear, OR set, XOR toggle, shifts.
- 4.5.10 Character vs numeric digit — ASCII 48 offset.
- 4.5.11 Images and sound — bitmap vs vector, sample rate, Nyquist, file-size formulas.
- 4.5.12 Compression — lossy vs lossless, RLE, Huffman, dictionary/LZ.
- 4.5.13 Encryption — Caesar, Vernam (OTP), symmetric vs asymmetric, hashes.